

Projeto 3 — SaaS de Gestão de Comércio (Java + Spring Boot)

1. Visão geral

Monólito modular multi-tenant em Java 21 + Spring Boot, com quatro módulos de negócio (E-commerce, Pedidos, PDV, Caixa/Admin) e um diferencial técnico: importação massiva de catálogo processada em paralelo com **Virtual Threads**. Contrasta de propósito com o Projeto 1 (microserviços distribuídos) — aqui a prova de competência é concorrência real dentro de um domínio centralizado, não coordenação entre serviços.

2. Arquitetura

2.1 Visão geral do projeto (deployment)

graph TB

Web[Frontend web] --> Mono

subgraph Mono["SaaS Monólito · Spring Boot · 1 processo"]

Auth[auth]

Tenant[tenant]

Catalog[catalog]

Order[order]

Ecommerce[ecommerce]

Pdv[pdv]

Cashier[cashier]

Payment[payment]

```
    Importing[importing]
```

```
end
```

```
    Mono --> DB[(MySQL)]
```

Diferente do Projeto 1, aqui existe **um único deployable**: um processo Spring Boot, um único banco. Não há rede entre módulos — toda comunicação interna é chamada de método direta, sem mensageria.

2.2 Diagrama de cada módulo (dependências internas)

graph LR

```
Auth[auth] -- valida token --> Tenant[tenant]
```

```
Tenant -- popula contexto --> Catalog[catalog]
```

```
Tenant -- popula contexto --> Order[order]
```

```
Catalog -- consulta produto --> Order
```

```
Order -- gera pedido --> Ecommerce[ecommerce]
```

```
Order -- gera pedido --> Pdv[pdv]
```

```
Order -- solicita cobrança --> Payment[payment]
```

```
Payment -- confirma pagamento --> Cashier[cashier]
```

```
Order -- fecha pedido --> Cashier
```

```
Importing[importing] -- upsert em lote --> Catalog
```

importing é o único módulo que escreve em **catalog** fora do fluxo normal de CRUD — é intencional, pra manter a fronteira de concorrência isolada num só lugar. **ecommerce** e **pdv** nunca se comunicam entre si diretamente; ambos delegam pra **order**, que é quem concentra a regra de negócio compartilhada.

2.3 Pipeline da importação paralela

graph TD

```
C[Cliente] -- upload CSV --> IC[ImportController]
```

```
IC -- cria job --> IJ[ImportJob: PENDING]

IJ -- 202 Accepted --> C

IJ -- dispara async --> CR[CsvChunkReader]

CR -- chunk ~500 linhas --> W1[ProductImportWorker]

CR -- chunk ~500 linhas --> W2[ProductImportWorker]

CR -- chunk ~500 linhas --> W3[ProductImportWorker]

W1 -- upsert em lote --> DB[(MySQL)]

W2 -- upsert em lote --> DB

W3 -- upsert em lote --> DB

W1 -- erro de linha --> AG[ImportResultAggregator]

W2 -- erro de linha --> AG

W3 -- erro de linha --> AG

AG -- persiste ImportJobError --> DB

AG -- consolida contadores --> IJ2[ImportJob: COMPLETED]

C -- GET status polling --> IJ2
```

Cada [ProductImportWorker](#) roda em sua própria virtual thread e com sua própria transação — uma falha em [W2](#) não derruba [W1](#) nem [W3](#). [ImportResultAggregator](#) é a única estrutura compartilhada entre as threads, e é thread-safe por design ([ConcurrentLinkedQueue](#)).

3. Estrutura de pacotes e classes

Pacote raiz sugerido: [com.portfolio.saas](#)

[com.portfolio.saas](#)

```
|—— config/
| |—— SecurityConfig.java      → cadeia de filtros HTTP, regras de autorização por rota
| |—— AsyncConfig.java        → bean do executor de virtual threads
(newVirtualThreadPoolTaskExecutor)
| |—— SpringDocConfig.java    → configuração do springdoc-openapi
|
|—— tenant/
| |—— Tenant.java             → entidade raiz do isolamento (id, name, document, planId,
isActive)
| |—— TenantRepository.java
| |—— TenantContext.java      → guarda o tenantId da requisição atual (RequestScope)
| |—— TenantFilter.java       → extrai tenantId do JWT e popula o TenantContext
|
|—— auth/
| |—— User.java               → entidade (id, tenantId, name, email, passwordHash, role)
| |—— UserRepository.java
| |—— AuthController.java     → POST /auth/login, /auth/register
| |—— AuthService.java
| |—— JwtService.java         → geração/validação de token
| |—— JwtAuthenticationFilter.java
|
|—— catalog/
| |—— Product.java           → entidade (sku, name, price, stockQuantity, categoryId)
```

	— Category.java	
	— ProductRepository.java	
	— CategoryRepository.java	
	— ProductController.java	→ CRUD REST padrão
	— ProductService.java	
	— order/	
	— Order.java	→ entidade (customerId, status, total, channel, createdAt)
	— OrderItem.java	
	— OrderChannel.java	→ enum: ECOMMERCE, PDV
	— OrderStatus.java	→ enum: PENDING, CONFIRMED, CANCELLED, COMPLETED
	— OrderRepository.java	
	— OrderService.java	→ regra de negócio compartilhada entre e-commerce e PDV
	— OrderController.java	
	— ecommerce/	
	— CartController.java	→ POST /cart/items, DELETE /cart/items/{id}
	— CartService.java	→ carrinho em sessão ou tabela temporária
	— CheckoutService.java	→ converte carrinho em Order (channel=ECOMMERCE)
	— pdv/	

	— RestaurantTable.java	→ entidade "mesa/comanda" (number, status, openedAt, closedAt)
	— TableRepository.java	
	— TableService.java	→ abrir/fechar comanda, adicionar item
	— TableController.java	
	— cashier/	
	— CashRegister.java	→ entidade de fechamento de caixa (openedAt, closedAt, totalCash, totalCard)
	— CashRegisterService.java	
	— ReportService.java	→ agregações para dashboards
	— ReportController.java	
	— payment/	
	— Payment.java	→ entidade (orderId, method, amount, status)
	— PaymentRepository.java	
	— PaymentService.java	
	— importing/	← MÓDULO DIFERENCIAL (paralelismo)
	— ImportJob.java	→ entidade (fileName, status, totalRows, processedRows, successCount, errorCount)
	— ImportJobError.java	→ entidade (importJobId, rowNum, rawData, errorMessage)

```

| | ImportStatus.java      → enum: PENDING, PROCESSING, COMPLETED,
COMPLETED_WITH_ERRORS, FAILED
| | ImportJobRepository.java
| | ImportJobErrorRepository.java
| | ImportController.java  → POST /imports (upload), GET /imports/{id}/status
| | ImportJobService.java  → cria o job, dispara o processamento assíncrono, atualiza
status final
| | CsvChunkReader.java    → leitura em streaming + divisão em chunks (~500 linhas)
| | ProductImportWorker.java → processa 1 chunk: valida, aplica regras, faz upsert em
lote
| | ImportResultAggregator.java → coleção thread-safe (ConcurrentLinkedQueue) de
sucessos/erros
| | ImportRetryPolicy.java  → configuração do Resilience4j para deadlock/timeout do
MySQL
|
| common/
| exception/
| | GlobalExceptionHandler.java → @RestControllerAdvice centralizado
| | ApiError.java
| | dto/
| | PageResponse.java
| | audit/
| | BaseEntity.java      → id, createdAt, updatedAt, @TenantId (mapeado via
Hibernate 6.4+)

```

Notas de design importantes:

- `BaseEntity` centraliza o campo de tenant usando a anotação `@TenantId` do Hibernate — evita reescrever `WHERE tenant_id = ?` manualmente em cada repositório.
- `order/` é compartilhado entre `ecommerce/` e `pdv/` — o `OrderChannel` é o que diferencia a origem do pedido, evitando duplicar entidade.
- `importing/` é o único módulo com preocupação de concorrência explícita. Nenhum outro módulo precisa lidar com isso — é assim que fica claro, em código, que o paralelismo é uma decisão pontual e justificada, não algo espalhado por todo o sistema.

4. Pipeline de funcionamento

3.1 Fluxo padrão — exemplo: criar um pedido

1. Requisição chega em `OrderController`, passando primeiro pelo `JwtAuthenticationFilter` e `TenantFilter`
2. `TenantContext` já está populado com o `tenantId` extraído do token
3. `OrderController` recebe o DTO, chama `OrderService`
4. `OrderService` valida regras (estoque disponível, preço atual) e persiste via `OrderRepository`
5. Hibernate aplica o filtro de tenant automaticamente graças ao `@TenantId` da `BaseEntity`
6. Resposta DTO é devolvida ao cliente

Esse fluxo é idêntico para os módulos de catálogo, PDV e caixa — só muda a entidade e a regra de negócio.

3.2 Fluxo do diferencial — importação massiva paralela

1. Cliente envia o CSV para `POST /imports` → `ImportController`
2. `ImportJobService` cria o registro `ImportJob` (status `PENDING`), grava no banco e devolve `202 Accepted` com o `jobId` — a requisição HTTP termina aqui, sem esperar o processamento
3. O processamento é disparado de forma assíncrona (submissão ao executor de virtual threads configurado em `AsyncConfig`)
4. `CsvChunkReader` lê o arquivo em streaming (nunca carrega tudo em memória) e agrupa as linhas em chunks de ~500 registros
5. Para cada chunk, uma `CompletableFuture` é criada rodando em uma virtual thread — como o trabalho é I/O-bound (esperando o MySQL), isso é praticamente gratuito em termos de recurso

6. Cada `ProductImportWorker`: valida os dados do seu chunk, aplica as regras de negócio, e executa um `INSERT ... ON DUPLICATE KEY UPDATE` em lote via `JdbcTemplate.batchUpdate` — upsert resolve idempotência automaticamente
 7. Se o MySQL devolver um erro transitório (deadlock, timeout), o `ImportRetryPolicy` (Resilience4j) tenta novamente com backoff exponencial antes de desistir daquele chunk
 8. Erros de linha ou chunk (validação ou falha definitiva) são adicionados ao `ImportResultAggregator`, uma coleção thread-safe — isso nunca interrompe as outras threads
 9. `CompletableFuture.allOf(...)` aguarda todos os chunks terminarem
 10. O `ImportResultAggregator` persiste os erros coletados em lote na tabela `ImportJobError`
 11. `ImportJobService` atualiza o `ImportJob` para `COMPLETED` ou `COMPLETED_WITH_ERRORS`, com os contadores finais
 12. O cliente consulta `GET /imports/{jobId}/status` (polling) e acompanha o progresso até o resultado final
-

5. Descrição para compartilhar com um amigo

Tô construindo um sistema de gestão completo pra loja/restaurante — tipo um mini-ERP: carrinho de compras, controle de pedidos, sistema de mesas/comandas pra quem trabalha com atendimento presencial, e um painel de caixa/admin pra fechar o dia. É multi-empresa (multi-tenant), ou seja, várias lojas diferentes podem usar o mesmo sistema com os dados totalmente isolados entre elas.

A parte que eu mais me orgulho é uma feature de importação em massa: o lojista sobe uma planilha com milhares de produtos de uma vez, e ao invés do sistema processar linha por linha (que ia demorar minutos), eu quebro esse arquivo em pedaços e processo tudo em paralelo, usando um recurso novo do Java (Virtual Threads) que deixa rodar milhares de tarefas concorrentes sem pesar no servidor. Se uma linha do arquivo tiver erro, ela é isolada e registrada — não trava a importação inteira.

É um projeto que mostra domínio ponta a ponta — front e back, um monólito bem estruturado com regras de negócio complexas, e processamento paralelo de verdade, não só CRUD básico.